



**UNIVERSIDAD
DE GRANADA**



Tecnologías Web

Grado en Ingeniería Informática

Programación de aplicaciones en el servidor con PHP (7)

Frameworks y herramientas habituales

Este documento está protegido por la Ley de Propiedad Intelectual (Real Decreto Ley 1/1996 de 12 de abril). Queda expresamente prohibido su uso o distribución sin autorización del autor.

(C) Javier Martínez Baena
jbaena@ugr.es
 Departamento de Ciencias de la Computación e Inteligencia Artificial
<http://decsai.ugr.es>



PHP y MVC
 Arquitectura basada en MVC

Conforme crece la aplicación ...

MVC facilita:

- Separación de conceptos
- Añadir nueva funcionalidad
- etc.

Existen otras tareas frecuentes en el desarrollo web:

- Control de accesos / privilegios
- Saneamiento de datos
- Enrutado
- Control de logs (bitácoras)
- Validación de formularios
- Internacionalización
- ...

Es frecuente disponer de algunos módulos para resolver estas tareas de forma simple

Framework PHP

- **Conjunto de bibliotecas/software de uso habitual en programación web**
- **Conjunto de reglas o estándares de trabajo**
 - Nomenclatura de funciones, clases, etc.
 - Jerarquías de clases
 - Estructura de carpetas
 - ...

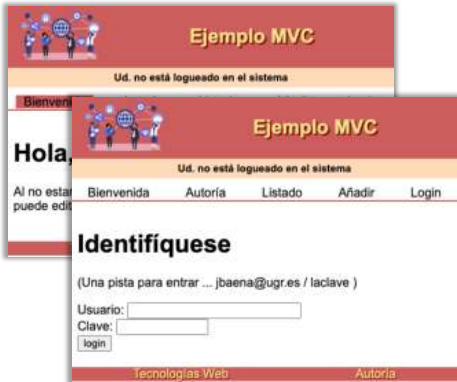
Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
2



PHP y MVC
 Arquitectura basada en MVC

Ejemplo

Mejora de diseño MVC + enrutado + control de accesos + ...



Listado de usuarios

Nombre	Email	
Javier Martínez	jbaena@ugr.es	Editar Borrar
Maria Martínez	maria@correo.web	Editar Borrar

Añadir nuevo usuario

Nombre:

Apellidos:

Email:

Tipo:

Clave:

[Enviar datos](#)

Edición de usuario

Nombre:

Apellidos:

Email:

Tipo:

Clave:

[Enviar datos](#)

<http://localhost:8888/tweb/tw2324/mvc/>
<http://localhost:8888/tweb/tw2324/mvc/login>

<http://localhost:8888/tweb/tw2324/mvc/user/list>
<http://localhost:8888/tweb/tw2324/mvc/user/add>
<http://localhost:8888/tweb/tw2324/mvc/user/edit/jbaena@ugr.es>



Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
3



UNIVERSIDAD
DE GRANADA

Tecnologías Web

Grado en Ingeniería Informática

PHP y la arquitectura MVC

1. MVC

- El Modelo: modelo abstracto y modelos concretos
- La Vista: vista abstracta y vistas concretas, plantillas
- El Controlador: modelo abstracto y modelos concretos

2. La petición HTTP

- Objeto Request
- Objeto Router (enrutamiento)

3. ... continúa ejemplo

4. Frameworks y bibliografía



DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena



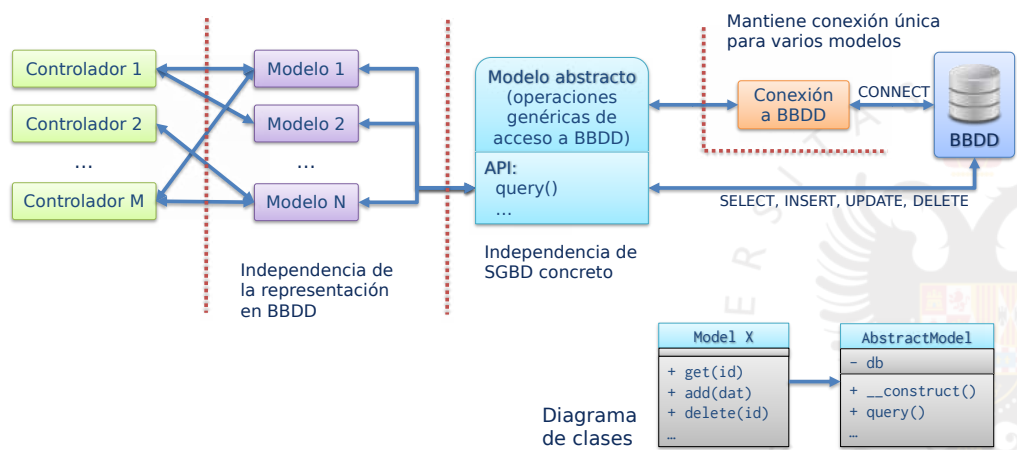
DECSAI

PHP y MVC

El Modelo

MVC - Implementación del modelo

Jerarquía de clases



Independencia de la representación en BBDD

Independencia de SGBD concreto

Mantiene conexión única para varios modelos

Conexión a BBDD

CONNECT

SELECT, INSERT, UPDATE, DELETE

Diagrama de clases



DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena



DECSAI

PHP y MVC

El Modelo / Modelo abstracto

MVC - Implementación del modelo - Modelo Abstracto

- Utiliza la API de PHP para acceder al SGBD (MySQLi, PDO, etc)
- Provee una API nueva a los modelos concretos de nuestra aplicación

```

abstract class AbstractModel {    // Clase abstracta
    protected $db;               // Recurso asociado a la BBDD (PDO)

    public function __construct() {
        $this->db = Database::getInstance(); // Conexión a la BBDD usando singleton
    }

    public function query($select,$params=[]) { // Consulta que debe devolver 1 tupla
        try {
            if (empty($params)) // Si no lleva parámetros -> consulta normal
                $pq = $this->db->query($select);
            else { // Si lleva parámetros -> consulta preparada
                $pq = $this->db->prepare($select);
                $pq->execute($params);
            }
            $result = $pq->fetch(PDO::FETCH_ASSOC);
        } catch (PDOException $e) {
            $result = null; // ... si hay error en la consulta
        }
        return $result;
    }
    ...

```

AbstractModel

- db

+ __construct()

+ query()

+ queryNoReturn()

+ queryAll()

+ getCurrentSchema()

+ columnCount()

+ tableExists()

...



DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena



PHP y MVC
 El Modelo / Modelo abstracto

MVC - Implementación del modelo - Modelo Abstracto

- Utiliza la API de PHP para acceder al SGBD (MySQLi, PDO, etc)
- Provee una API nueva a los modelos concretos de nuestra aplicación

```

...
// Comprueba la existencia de una tabla
public function tableExists($tab) {
    $existe = $this->query("SELECT COUNT(*) AS C FROM information_schema.tables
        WHERE table_schema='".$this->getCurrentSchema()."' AND
        table_name='".$tab."'");
    return ($existe['C']==0) ? false : true;
}

// Devuelve el número de columnas de una tabla
public function columnCount($tab) {
    $q = $this->query("SELECT COUNT(*) AS C FROM information_schema.columns
        WHERE table_name='".$tab."'");
    return $q['C'];
}
...
        
```

AbstractModel
 - db
 + __construct()
 + query()
 + queryNoReturn()
 + queryAll()
 + getCurrentSchema()
 + columnCount()
 + tableExists()
 ...

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
7



PHP y MVC
 El Modelo / Modelo concreto

MVC - Implementación del modelo - Modelo Concreto (MUsuario)

- Utiliza la API del modelo abstracto para hacer consultas
- Provee una API nueva a los controladores de nuestra aplicación independiente del SGBD (orientada a la aplicación)

```

class MUsuario extends AbstractModel {

    function __construct() {
        parent::__construct();
        if (!$this->tableExists('usuarios'))
            $this->createTable();
    }

    public function get($email) {
        $r = $this->query('SELECT * FROM usuarios WHERE email=:email', ['email'=>$email]);
        return empty($r) ? null : $r;
    }
}
...
        
```

AbstractModel

↑

MUsuario

+ __construct()
 + get()
 + delete()
 + getAll()
 + update()
 + setPassword()
 + add()
 + createTable()
 ...

Un modelo concreto se centra en manipular una tabla o vista de la BBDD (o conjunto de tablas)

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
8



PHP y MVC
 El Modelo / Modelo concreto

MVC - Implementación del modelo - Modelo Concreto (MUsuario)

- Utiliza la API del modelo abstracto para hacer consultas
- Provee una API nueva a los controladores de nuestra aplicación independiente del SGBD (orientada a la aplicación)

```

public function createTable() {
    if (!$this->tableExists('usuarios')) {
        $q = "CREATE TABLE IF NOT EXISTS usuarios (
            `email` VARCHAR(100) NOT NULL,
            `nombre` VARCHAR(45) NULL,
            `apellidos` VARCHAR(100) NULL,
            PRIMARY KEY (`email`))
            ENGINE = InnoDB;";
        $rr = $this->db->query($q);
    }
}
        
```

AbstractModel

↑

MUsuario

+ __construct()
 + get()
 + delete()
 + getAll()
 + update()
 + setPassword()
 + add()
 + createTable()
 ...

Alternativas:

- Crear las tablas fuera de la aplicación
- Crear las tablas desde la aplicación en su primera ejecución

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
9

PHP y MVC
 El Modelo

MVC - El modelo

- Modelo abstracto: Independencia entre aplicación y API de SGBD
Provee API a todos los modelos concretos para acceder al SGBD
- Modelo concreto: sigue asumiendo SQL como lenguaje de consulta y provee API al controlador independiente de SQL
- Controlador usa API del modelo concreto y no usa SQL

Ind SQL
Ind SGBD

Controlador

Ind SGBD

Modelo concreto

Modelo abstracto

BBDD

```

$this->mduser = new MUsuario();
$data = $this->mduser->get('jbaena@ugr.es');

public function get($email) {
    $r = $this->query("SELECT * FROM usuarios WHERE email=:email
                     ORDER BY apellidos DESC",[email]='>$email]);

public function query($select,$params=[]) {
    $pq = $this->db->prepare($select);
    $pq->execute($params);

```

API de PHP para acceso a BBDD con PDO

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 10

PHP y MVC
 El Modelo / ORM

MVC - El modelo abstracto

Es frecuente que el modelo abstracto provea interfaz para independizar al modelo concreto del lenguaje de consulta del SGBD (SQL, ...)

Modelo concreto

```

public function get($email) {
    $r = $this->query("SELECT * FROM usuarios WHERE email=:email
                     ORDER BY apellidos DESC",[email]='>jbaena@ugr.es']);

    ↓

    $r = $this->select()
        ->from("usuarios")
        ->where("email=?", "jbaena@ugr.es")
        ->order("apellidos", "desc")
        ->all();

```

Provee una API más sofisticada para evitar que el desarrollador de la aplicación conozca lenguajes de consulta de BBDD (SQL, etc)

ORM (Object-relational mapping)

Capa que mapea objetos en PHP a tablas/vistas/... en la BBDD
Ejemplos: Propel, Eloquent (Laravel), Doctrine DBAL, ...

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 11

Tecnologías Web
 Grado en Ingeniería Informática
PHP y la arquitectura MVC

UNIVERSIDAD DE GRANADA

1. El Modelo
- » 2. La Vista
3. El Controlador
4. La petición HTTP y el enrutamiento
5. ... continúa ejemplo
6. Frameworks y bibliografía

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena



PHP y MVC
 La Vista

MVC - La vista

- Sistema de plantillas:
 - Diseño de las páginas en HTML+marcas y sustitución de marcas con PHP
 - Sistemas de plantillas de terceros ← **Muy frecuente**
- Código HTML generado con PHP ← **En este ejemplo**

Ejemplo: generando la página con PHP

```

abstract class AbstractView {
    protected $request, $login; // Información de interés
    protected $web; // Estructura de datos

    public function __construct($req,$login) {
        $this->request = $req;
        $this->login = $login;
        // Crear estructura web
        $this->web = [];
        $this->web['head'] = new HTMLHead();
        $this->web['head']->setBase($this->preURL().'');
        $this->web['body'] = new HTMLTag_body();
        $this->web['html'] = new HTMLTag_html();
        $this->web['html']->addContent($this->web['head']);
        $this->web['html']->addContent($this->web['body']);
    }
  
```

AbstractView
 - web
 + __construct()
 + render()
 + setPageTitle()
 + addCSSLink(...)
 + addMeta()
 ...

↑

WebPage
 + addLoginForm(...)
 + addUserList(...)
 ...

Estructura genérica de cualquier web

 Estructura de la web concreta que estamos creando

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
13



PHP y MVC
 La Vista

MVC - La vista abstracta

Dispone una infraestructura común a cualquier web que queramos crear

```

public function render() { // Renderizamos contenido de página
    return '<!DOCTYPE html>'.PHP_EOL.$this->web['html']->render();
}

public function setPageTitle($tit) {
    $this->web['head']->setTitle($tit);
}

public function addCSSLink($css) {
    $this->web['head']->addCSS($css);
}

public function addMeta($n,$c) {
    $this->web['head']->addMeta($n,$c);
}
  
```

AbstractView
 - web
 + __construct()
 + render()
 + setPageTitle()
 + addCSSLink(...)
 + addMeta()
 ...

\$this->web

- Almacena representación jerárquica de la página (objetos que representan tags HTML)
- Los controladores añaden, quitan o modifican elementos de esta estructura
- El controlador finalizará devolviendo el resultado de renderizar

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
14

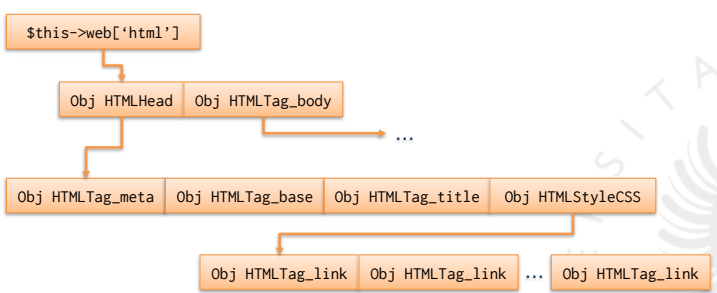


PHP y MVC
 La Vista

MVC - La vista abstracta

Estructura básica de cualquier web que se compone de objetos:

- Tags HTML
- Objetos compuestos (grupos de otros objetos o tags HTML)



```

graph TD
    A["$this->web['html']"] --> B["Obj HTMLHead"]
    A --> C["Obj HTMLTag_body"]
    C --> D["Obj HTMLTag_meta"]
    C --> E["Obj HTMLTag_base"]
    C --> F["Obj HTMLTag_title"]
    C --> G["Obj HTMLStyleCSS"]
    F --> H["Obj HTMLTag_link"]
    F --> I["Obj HTMLTag_link"]
    F --> J["Obj HTMLTag_link"]
  
```

Todos los objetos disponen de un método `render()` para generar el correspondiente código HTML que permite generar el código completo de la página de forma recursiva.

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
15



PHP y MVC

La Vista

MVC - La vista abstracta

PHP y código generado a partir de la clase abstracta (en este ejemplo no sería abstracta para poder instanciarla)

```

$webpage = new AbstractView($request);
$webpage->addCSSLink($webpage->createUrl('public/css/estilo.css'));
$webpage->addMeta('author', 'Javier');
$webpage->setPageTitle('Ejemplo MVC');
print($webpage->render());

```

↓

```

<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta name="author" content="Javier" />
    <title>Ejemplo MVC</title>
    <link href="http://localhost:8888/tweb/tw2324/mvc/public/css/estilo.css" rel="stylesheet" type="text/css" />
    <base href="http://localhost:8888/tweb/tw2324/mvc/" />
  </head>
  <body>
  </body>
</html>

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

16



PHP y MVC

La Vista

Continuación del ejemplo con PHP: vista concreta

Vista abstracta: provee marco común para generar vista
 Vista concreta: personaliza la página que estamos creando

```

class WebPage extends AbstractView {
    // Menú de navegación
    protected $nav_menu, $nav_active;

    public function __construct($req,$login) {
        parent::__construct($req,$login);
        $this->createNewPage($req);
    }

    public function render() {
        if (isset($this->nav_active))
            $this->web['navigation']->setActive($this->nav_active);
        return parent::render();
    }
}

```

```

public function createNewPage($req) {
    // Definir menú de navegación (se podría cargar de JSON)
    $this->nav_menu = [ ['Bienvenida','id_nav_index'],
                        ['Autoría','autoria','id_nav_autoria'],
                        ['Listado','user/list','id_nav_ulist'],
                        ['Añadir','user/add','id_nav_add'] ];

    if ($this->login->isLoggedIn()) {
        $this->nav_menu[] = ['Salir','unlog','id_mav_log'];
    } else {
        $this->nav_menu[] = ['Login','login','id_mav_log'];
    }

    // Crear elementos de la web
    $this->web['header'] = new HTMLHeader(['class'=>'miencabezad
    $this->web['footer'] = new HTMLFooter(['class'=>'mipiedepagi
    $this->web['contenidos'] = new HTMLTag_div(['id'=>'misconten
    $this->web['main'] = new HTMLTag_main();
    $this->web['aside'] = new HTMLASide();
    ...
}

```

AbstractView

- web
 + __construct()
 ...

WebPage

+ __construct()
 + addMain()
 + addAsideItem()
 + addFooterItem()
 + addMessage()
 + addLoginForm(...)
 + addUserList(...)
 + addUserEdit()
 ...

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

17



PHP y MVC

La Vista

Continuación del ejemplo con PHP: vista concreta

Vista abstracta: provee marco común para generar vista
 Vista concreta: personaliza la página que estamos creando

```

class HTMLHeader extends HTMLBlock {
    protected $img, $titulo, $subtitulo;

    public function __construct($atr='') {
        parent::__construct();
        $this->img = new HTMLTag_img();
        $this->titulo = new HTMLTag_h1();
        $this->subtitulo = new HTMLTag_h2();

        $this->block[] = new HTMLTag_header($atr, [ $this->img,
                                                    new HTMLTag_div('', $this->titulo),
                                                    $this->subtitulo ]);
    }

    public function setImage($i) { $this->img->setAttr('src',$i); }
    public function setTitle($t) { $this->titulo->setContent($t); }
    public function setSubtitulo($t) { $this->subtitulo->setContent($t); }
}

```

Objeto contenedor del header del documento

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

18



PHP y MVC
 La Vista

Sistemas de plantillas

- Alternativa a la generación de código desde PHP.
- Diseño de las páginas con HTML incluyendo macros en donde deberían ir datos concretos.
- Ejemplos: Twig, Plates, Smarty, ...

Un ejemplo con Twig:

layout.twig

```
<html>
<head>
  <meta charset="UTF-8" />
  <base href="{% block base %}{% endblock %}" />
  <title>{% block title %}{% endblock %}</title>
  {% block css %}{% endblock %}
</head>
<body>
  {% block content %}{% endblock %}
</body>
</html>
```

{% block xxx %}{% endblock %} Desde PHP se sustituye el valor

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
19



PHP y MVC
 La Vista

Sistemas de plantillas

Ejemplo con Twig

mipagina.twig

```
{% extends 'layout.twig' %}

{% block title %}
{{ titulo }}
{% endblock %}

{% block base %}
{{ base }}
{% endblock %}

{% block css %}
{% for item in csslist %}
<link href="{{ item }}" rel="stylesheet" type="text/css"/>
{% endfor %}
{% endblock %}
...
```

web.php

```
$params = ['titulo'=>'Mi página',
           'base'=>'http://localhost/mvc',
           'csslist'=>['http://localhost/mvc/views/css/estilo.css']];
echo $twig->loadTemplate('mipagina.twig')->render($params);
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
20



Tecnologías Web
 Grado en Ingeniería Informática
PHP y la arquitectura MVC

1. El Modelo
2. La Vista
3. El Controlador
4. La petición HTTP y el enrutamiento
5. ... continúa ejemplo
6. Frameworks y bibliografía


Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena

PHP y MVC
El Controlador

El controlador
Seguimos el mismo esquema: controlador abstracto + controladores concretos

```
abstract class AbstractController {
    protected $request, $login, $page;

    public function __construct($req,$login,$met=null,$par=null,$webpage=null) {
        $this->request = $req;
        $this->login = $login;
        // Crear estructura de página básica
        if ($webpage===null and !isset($this->page))
            die('AbstractController: no está definida la página web');
        if ($webpage!=null)
            $this->page = $webpage;

        if (is_string($met) and $met!='') {
            if ($par) // Comprobar si se le pasan parámetros o no
                $this->$met($par);
            else
                $this->$met();
        }
    }

    public function render() {
        return $this->page->render();
    }
}
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 22

PHP y MVC
El Controlador

El controlador
Seguimos el mismo esquema: controlador abstracto + controladores concretos

```
class CInfo extends AbstractController {

    public function __construct($req,$login,$met,$par=null,$webpage=null) {
        parent::__construct($req,$login,$met,$par,$webpage);
    }

    public function noRoute() {
        $this->page->addMessage('La ruta no está definida','h1');
    }

    public function autoria() {
        $this->page->setNavActive('id_nav_autoria');
        $this->page->addMessage('Autoria de la aplicación','h1');
        $this->page->addMessage('Javier Martínez','p');
        $this->page->addMessage('Ejemplo de aplicación MVC para Tecnologías Web','p');
        $this->page->addMessage('Imágenes de ejemplo de Freepik','p');
    }
}
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 23

PHP y MVC
El Controlador

El controlador
Seguimos el mismo esquema: controlador abstracto + controladores concretos

```
class CUsuario extends AbstractController {
    protected $mduser;

    public function __construct($req,$login,$met,$par=null,$webpage=null) {
        $this->mduser = new MUsuario();
        parent::__construct($req,$login,$met,$par,$webpage);
    }

    public function list() {
        $this->page->setNavActive('id_nav_ulist');
        $list = $this->mduser->getAll();
        foreach ($list as $u)
            $this->page->addUser2List($u['nombre'],$u['apellidos'],$u['email'],
                'Editar','user/edit/:'.$u['email'],
                'Borrar','user/delete/:'.$u['email']);
    }
}
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 24



Tecnologías Web

Grado en Ingeniería Informática

PHP y la arquitectura MVC

1. El Modelo
2. La Vista
3. El Controlador
- » 4. La petición HTTP y el enrutamiento
5. ... continúa ejemplo
6. Frameworks y bibliografía



Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena



PHP y MVC

Herramientas adicionales

Herramientas de uso habitual en este tipo de sistemas

- Request: objeto que almacena datos de la petición HTTP (encapsula \$_GET, \$_POST, \$_SERVER, ...)
- Session: objeto que gestiona las sesiones (encapsula el array \$_SESSION)
- Login: objeto que controla el logueo de usuarios
- Router: objeto que, a partir de un objeto Request, determina el controlador, acción y parámetros de la petición
- ...



Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

26



PHP y MVC

El objeto Request

El objeto Request

Almacena y gestiona la información recibida en la petición (HTTP, Query string, ...)

```

class Request {
    protected $root;
    protected $domain, $method, $path, $scheme;
    protected $parameters, $query, $parametersroute;

    public function __construct() {
        Session::sessionStart();
        $this->method = $_SERVER['REQUEST_METHOD'];
        $this->domain = $_SERVER['HTTP_HOST'];
        $this->scheme = ((isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') ||
            $_SERVER['SERVER_PORT'] == 443) ? 'https' : 'http';
        $this->query = $_SERVER['QUERY_STRING'];
        $p = strpos($_SERVER['SCRIPT_NAME'], 'index.php');
        $this->root = substr($_SERVER['SCRIPT_NAME'], 1, $p-2);
        $this->path = $_SERVER['REQUEST_URI'];
        if (strpos($this->path, $this->root) == 1) // Quitar root path. (pos==1 porque hay que contar el / del comienzo)
            $this->path = substr($this->path, strlen($this->root)+1); // incluimos el / inicial de root y el que hay entre root
        if ($this->query != '') { // Quitamos el query string
            $p = strpos($this->path, $this->query);
            $this->path = substr($this->path, 0, $p-1);
        }
        $this->parameters = array_merge($_POST, $_GET);
    }
}

```



Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

27



PHP y MVC
 El objeto Request

El objeto Request

Integra el saneado de datos de POST/GET

```

public function getParam($p) {
    if (($p!=null) and array_key_exists($p,$this->parameters)) {
        if (is_array($this->parameters[$p]))
            return $this->getParamArray($this->parameters[$p]);
        else
            return htmlspecialchars($this->parameters[$p]);
    } else
        return null;
    }

protected function getParamArray($r) {
    if (is_array($r)) {
        if (count($r)=1)
            return [$this->getParamArray($r[0])];
        else
            return array_merge([$this->getParamArray($r[0])], $this->getParamArray(array_slice($r,1)));
    } else
        return htmlspecialchars($r);
    }
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
28



PHP y MVC
 El objeto Request

El objeto LoginStatus

Mantiene información sobre el estado del usuario logueado (si está logueado, datos, ...)

```

class LoginStatus {
    protected $request;

    public function __construct($req) {    $this->request = $req;    }

    // ***** Logear usuario: se añaden todos los datos de $dat
    public function login($dat) {
        Session::unsetSession('login');
        if (is_array($dat))
            Session::setSession('login',$dat);
    }
    public function unlogin() {    Session::unsetSession('login');    }

    public function isLoggedIn() {    return Session::getSession('login') ? true : false;    }

    public function getData($k) { ... }
    public function setData($k,$v) { ... }
}
  
```

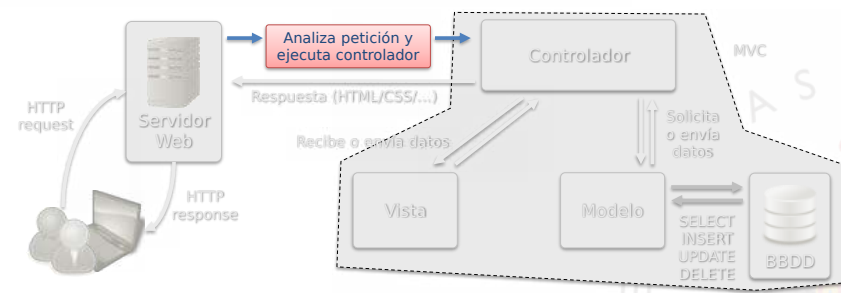
Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
29



PHP y MVC
 Herramientas adicionales

El objeto Router (enrutador)

- Se integra en el front-controller
- Analiza la petición y determina el controlador+método a ejecutar



Front-controller (index.php)

```

$req = new Request();
$login = new LoginStatus($req);
$rout = new Router($req,$login);
$webpage = new WebPage($req,$login);
$c = $rout->execute($webpage);
print($c->render());

// Obtener HTTP request
// Crear objeto con estado de login
// Crear objeto para enrutado
// Crear web básica
// Ejecutar el controlador solicitado
// Devolver resultado (HTML)
      
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
30



PHP y MVC
 El objeto Router

El objeto Router

Analiza la URI y:

- Determina la tarea a realizar (en caso de MVC: controlador + acción)
- Obtiene parámetros incluidos en la URI (distintos de query string)

Alternativas para acceder a las páginas de la aplicación:

- Un script .php para cada tarea/página
- Un script .php único para toda la aplicación:
 - Acción determinada por parámetros en query string
 - Acción determinada por parámetros en URI

➡ Un script .php para cada tarea:

```
http://localhost/app/user/list.php
http://localhost/app/user/delete.php
http://localhost/app/ventas/informe/resumen.php?formato=breve
```

- Cada script .php debe mantener una estructura común (copia/pega)
- Añadir funcionalidad → añadir nuevos scripts
- URL muy ligada a estructura de carpetas del sitio

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
31



PHP y MVC
 El objeto Router

El objeto Router

Analiza la URI y:

- Determina la tarea a realizar (en caso de MVC: controlador + acción)
- Obtiene parámetros incluidos en la URI (distintos de query string)

➡ Un script .php único para toda la aplicación. Acción y parámetros en QS:

```
http://localhost/app/index.php?action=list
http://localhost/app/index.php?action=delete&user=pepito
http://localhost/app/index.php?action=ventas&subaction=resumen&formato=breve
```

- Hay menos scripts que mantener. Mejor control de errores. Evita copia/pega
- Oculta estructura de carpetas y ficheros internos
- Los parámetros en query string → URL compleja
- ¿Petición indebidas? (URL no existente, ficheros internos, etc)

➡ Un script .php único para toda la aplicación. Acción y parámetros en URI + QS:

```
http://localhost/app/user/list
http://localhost/app/user/:pepito/delete
http://localhost/app/ventas/resumen?formato=breve
```

- URL más amigable y fácil de recordar
- Oculta más detalles de implementación (lenguaje/carpetas)
- Mejor control de peticiones indebidas de usuarios

Habitual

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
32



PHP y MVC
 El objeto Router

¿Cómo redirigir todas las peticiones a un script?

La redirección la hace el servidor web

- Apache: módulo **mod_rewrite**
 - https://httpd.apache.org/docs/current/mod/mod_rewrite.html
 - https://www.digitalocean.com/community/tutorials/how-to-rewrite-urls-with-mod_rewrite-for-apache-on-ubuntu-22-04
- NGINX:
 - <https://www.nginx.com/blog/creating-nginx-rewrite-rules/>
 - <https://www.nginx.com/blog/creating-nginx-rewrite-rules/>

Para usar mod_rewrite en Apache:

- Activar módulo en servidor: `a2enmod mod_rewrite`
- Habilitar su uso (en config o .htaccess): `RewriteEngine on`

Expresión regular para localizar un patrón en la URL

- a partir del directorio en donde se define la regla
- y hasta antes del query string

RewriteRule pattern substitution [flags]

↖ Lo que hay que poner en lugar del patrón localizado

↗ Posibles opciones:

- No sensible a mayus/minus
- Escapar algunos caracteres
- Descartar o no el QS
- ...

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
33

PHP y MVC
El objeto Router

`http://localhost/rewrite/verdatos.php`
Script PHP que muestra algunos datos de \$_SERVER y \$_GET

`http://localhost/rewrite/.htaccess`
`RewriteRule ^info$ verdatos.php`

`http://localhost/rewrite/verdatos.php`

SERVER

HTTP_HOST = localhost
REQUEST_METHOD = GET
QUERY_STRING =
REQUEST_URI = /rewrite/verdatos.php
SCRIPT_NAME = /rewrite/verdatos.php

`http://localhost/rewrite/info`

SERVER

HTTP_HOST = localhost
REQUEST_METHOD = GET
QUERY_STRING =
REQUEST_URI = /rewrite/info
SCRIPT_NAME = /rewrite/verdatos.php

`http://localhost/rewrite/info`
... se redirige a ...
`http://localhost/rewrite/verdatos.php`

`http://localhost/rewrite/info?var=23`

SERVER

HTTP_HOST = localhost
REQUEST_METHOD = GET
QUERY_STRING = var=23
REQUEST_URI = /rewrite/info?var=23
SCRIPT_NAME = /rewrite/verdatos.php

GET

`var = 23`

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 34

PHP y MVC
El objeto Router

`http://localhost/rewrite/.htaccess`
`RewriteRule ^miapp verdatos.php` ¡ Cuidado: eliminado el \$ del patrón !

`http://localhost/rewrite/miapp/otra/pagina/mas?var=23`

SERVER

HTTP_HOST = localhost
REQUEST_METHOD = GET
QUERY_STRING = var=23
REQUEST_URI = /rewrite/miapp/otra/pagina/mas?var=23
SCRIPT_NAME = /rewrite/verdatos.php

GET

`var = 23`

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 35

PHP y MVC
El objeto Router

`http://localhost/rewrite/.htaccess`
`RewriteRule ^miapp/([a-z]+) verdatos.php?color=$1 [QSA]`

Subexpresiones del patrón se pueden utilizar en la sustitución

`http://localhost/rewrite/miapp/rojo?x=3`

SERVER

HTTP_HOST = localhost
REQUEST_METHOD = GET
QUERY_STRING = color=rojo&x=3
REQUEST_URI = /rewrite/miapp/rojo?x=3
SCRIPT_NAME = /rewrite/verdatos.php

GET

`color = rojo`
`x = 3`

- Al incluir QS en la sustitución, por defecto se anula el QS original si lo hubiese
- QSA: indica que añada el QS original si hubiese

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 36



PHP y MVC
 El objeto Router

Aplicar reglas de forma condicional

`RewriteCond teststring condition [flags]`

Variable con alguna característica de la petición
Expresión regular con la que debe hacer matching la variable

Si teststring hace matching con condition entonces se ejecuta la regla RewriteRule que haya a continuación.

http://localhost/rewrite/.htaccess

`RewriteCond %{QUERY_STRING} prueba`
`RewriteRule ^miapp verdatos.php`

QUERY_STRING = prueba *http://localhost/rewrite/miapp/rojo?prueba*

REQUEST_URI = /rewrite/miapp/rojo?prueba

SCRIPT_NAME = /rewrite/verdatos.php

GET

prueba =

Not Found *http://localhost/rewrite/miapp/rojo*

The requested URL /rewrite/miapp/rojo was not found on this server.

Apache/2.4.29 (Ubuntu) Server at localhost Port 80

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

37



PHP y MVC
 El objeto Router

Variables disponibles para el teststring

HTTP headers: HTTP_ACCEPT HTTP_COOKIE HTTP_FORWARDED HTTP_HOST HTTP_PROXY_CONNECTION HTTP_REFERER HTTP_USER_AGENT	connection & request: AUTH_TYPE CONN_REMOTE_ADDR CONTEXT_PREFIX CONTEXT_DOCUMENT_ROOT PATH_INFO QUERY_STRING REMOTE_ADDR REMOTE_HOST REMOTE_IDENT REMOTE_PORT REMOTE_USER REQUEST_METHOD SCRIPT_FILENAME	server internals: DOCUMENT_ROOT SCRIPT_GROUP SCRIPT_USER SERVER_ADDR SERVER_ADMIN SERVER_NAME SERVER_PORT SERVER_PROTOCOL SERVER_SOFTWARE	date and time: TIME_YEAR TIME_MON TIME_DAY TIME_HOUR TIME_MIN TIME_SEC TIME_WDAY TIME	specials: API_VERSION CONN_REMOTE_ADDR HTTPS IS_SUBREQ REMOTE_ADDR REQUEST_FILENAME REQUEST_SCHEME REQUEST_URI THE_REQUEST
---	--	---	--	--

https://httpd.apache.org/docs/2.4/mod/mod_rewrite.html
<https://httpd.apache.org/docs/2.4/expr.html#vars>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

38



PHP y MVC
 El objeto Router

Un ejemplo de Router

Definir todas las rutas del sitio: Para cada URI se establece un controlador y una acción así como otros datos de interés (privilegios, parámetros, etc)

```

{
  "": {
    "controller": "CInfo",
    "method": "hello",
    "privileges": []
  },
  "welcome": {
    "controller": "CInfo",
    "method": "hello",
    "privileges": []
  },
  "autoria": {
    "controller": "CInfo",
    "method": "autoria",
    "privileges": []
  },
  "user/edit/:email": {
    "controller": "CUsuario",
    "method": "edit",
    "privileges": ["admin"]
  },
  "user/edit/:email/confirm": {
    "controller": "CUsuario",
    "method": "editConfirm",
    "privileges": ["admin"]
  },
  "user/edit/:email/confirm/info": {
    "controller": "CUsuario",
    "method": "editConfirmInfo",
    "privileges": ["admin"]
  },
  "user/list": {
    "controller": "CUsuario",
    "method": "list",
    "privileges": ["user", "admin"]
  },
  "login": {
    "controller": "CLogin",
    "method": "login",
    "privileges": []
  },
  "login/validate": {
    "controller": "CLogin",
    "method": "validate",
    "privileges": []
  },
  ...
}

```

Fichero config.json para definir todas las rutas y acciones

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

39



PHP y MVC
 El objeto Router

Un ejemplo de Router

Definir todas las rutas del sitio: Para cada URI se establece un controlador y una acción así como otros datos de interés (privilegios, parámetros, etc)

```

class Router {
    protected $rutas; // Lista de rutas disponibles en la aplicación
    protected $request; // Objeto con datos de HTTP request
    protected $login; // Objeto con datos de logeado del usuario

    public function __construct($req,$login) {
        $this->request = $req;
        $this->login = $login;
        // Cargar configuración de enrutado
        $json = file_get_contents(__DIR__.'../config/routes.json');
        $this->rutas = json_decode($json, true);
        if (!$this->rutas)
            die('Error fatal en decodificación de rutas de JSON');
    }
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

40



PHP y MVC
 El objeto Router

Un ejemplo de Router

Dada una petición (objeto Request) determinar la ruta asociada

```

public function locateRoute($url=null) {
    $request_div = ($url===null) ? explode('/', $this->request->getPath()) : explode('/', $url);
    // Recorremos lista de rutas de la configuración y buscamos la solicitada
    foreach ($this->rutas as $ruta => $datos) {
        $ruta_div = explode('/', $ruta); // Ruta para comparar con
        $encontrada = true;
        $par = []; // En este array almacenaremos los parámetros de la ruta
        if (count($ruta_div)==count($request_div)) { // Si tienen igual número de items separados por /
            // Comparamos items de la ruta uno a uno
            for ($i=0; $encontrada and $i<count($ruta_div); $i = $i+1) {
                $match = Router::compare_items($request_div[$i], $ruta_div[$i], $datos['params']??null);
                if ($match===false)
                    $encontrada = false;
                else { if ($match!==true) $par[$match['nombre']] = $match['valor']; }
            }
        } else $encontrada = false; // No tienen igual número de items -> no son iguales
        if ($encontrada) { // Se ha localizado la ruta
            $this->rutas[$ruta]['params'] = [];
            foreach ($par as $k=>$v)
                $this->rutas[$ruta]['params'][$k] = $v;
            return $this->rutas[$ruta];
        }
    }
    return null; }
  
```

Compara dos ítems y devuelve:
 - false si no coinciden
 - true si coinciden
 - En caso de que se trate de un parámetro devuelve el nombre y el valor del mismo

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

41



PHP y MVC
 El objeto Router

Un ejemplo de Router

Ejecutar el método/controlador solicitado por el usuario

```

public function execute($webpage) {
    $action = $this->locateRoute(); // Localizar el controlador solicitado
    if ($action===null) {
        require_once(__DIR__ . "../controllers/CInfo.php");
        $c = new CInfo($this->request, $this->login, 'noRoute', null, $webpage);
    } else {
        $this->request->setParamsRoute($action['params']??null); // Añadir a request parámetros de ruta y privilegios
        if (isset($action['privileges'])) $this->request->setParamsRoute(['privileges'=>$action['privileges']]);
        // Comprobar privilegios y determinar acción: logueo, continuar, redirigir ...
        if (!empty($action['privileges']) && !in_array($this->login->getData('userprivs'), $action['privileges'])) {
            $c = new CInfo($this->request, $this->login, 'noPermission', null, $webpage);
            return $c; // Finalizar con error
        }
        @include_once(__DIR__ . "../controllers/{$action['controller']}.php"); // Intentar incluir el fichero del controlador
        // Y comprobar si existe el controlador y el método
        if (class_exists($action['controller']) && method_exists($action['controller'], $action['method'])) {
            $c = new $action['controller']($this->request, $this->login, $action['method'], null, $webpage);
        } else { // No existe el controlador
            require_once(__DIR__ . "../controllers/CInfo.php");
            if (!class_exists($action['controller'])) $c = new CInfo($this->request, $this->login, 'noController', null, $webpage);
            else $c = new CInfo($this->request, $this->login, 'noAction', null, $webpage);
        }
    }
    return $c; }
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

42



PHP y MVC
 La aplicación

El script principal index.php

1. Crea los objetos principales: Request, LoginStatus, Página web, Router
2. Ejecuta el controlador
3. Devuelve el resultado a apache

RewriteEngine On *Fichero .htaccess para redirigir todas las peticiones a index.php*

RewriteCond %{REQUEST_FILENAME} !-f # Si no es un fichero existente -> redirigir a index.php

RewriteRule ^ index.php [L] # [L] no sigue aplicando reglas posteriores

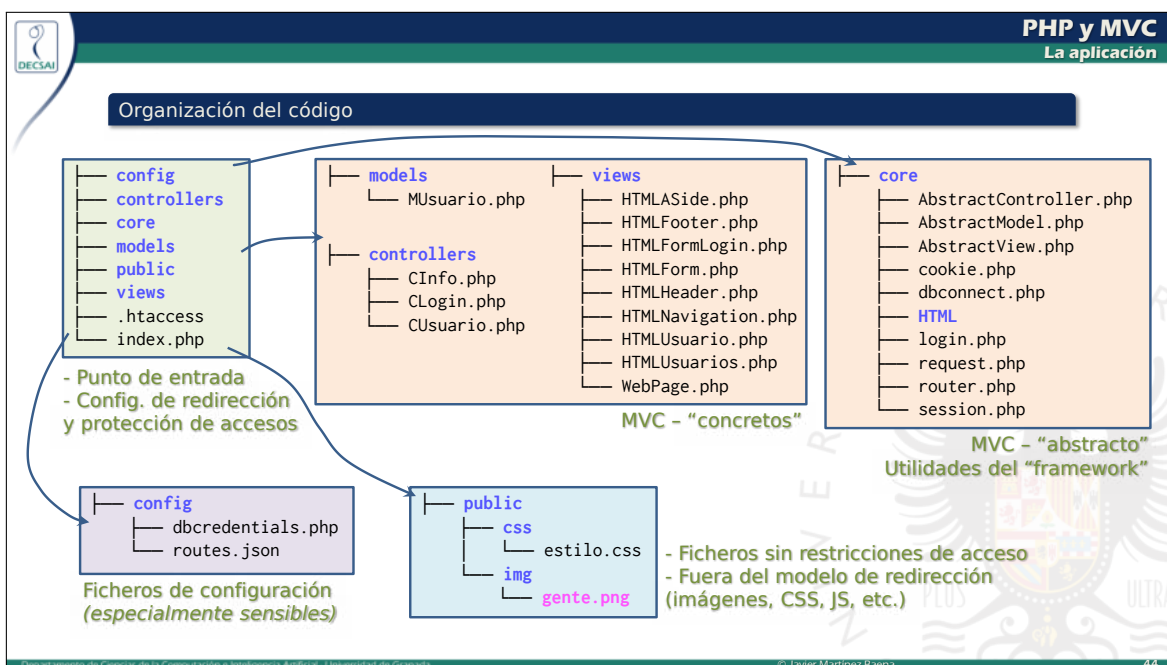
RewriteRule !public index.php [L] # Si no comienza por public redirigir a index.php

```

$req = new Request();           // Obtener HTTP request
$log = new LoginStatus($req);   // Crear objeto con estado de login
$rout = new Router($req,$log);  // Crear objeto para enrutado
$webpage = new WebPage($req,$log); // Crear web
$c = $rout->execute($webpage);  // Ejecutar el controlador solicitado
print($c->render());            // Devolver resultado (HTML)
      
```

Fichero index.php

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
43





PHP y MVC
 La aplicación

Detalle de interés en el ejemplo

- Paso de información entre páginas (caso "Editar usuario"):
 - Campos hidden
 - Campos readonly/disabled
 - Caso de ficheros/imágenes
 - Uso de variables de sesión
- Añadir nueva funcionalidad (no hay que tocar ficheros en core/):
 - Crear nuevo class modelo (fichero en models/) [si procede]
 - Crear nuevo class controlador (fichero en controllers/) [si procede]
 - Añadir nuevos métodos
 - Ampliar class WebPage o añadir nuevas clases en views/
 - Añadir nuevas rutas en config/routes.json
- Seguridad:
 - Redirección a index.php
 - Impedir acceso a carpetas y ficheros .php o de cualquier otro tipo (excepto a public/)

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
45



PHP y MVC
Frameworks de PHP

Frameworks de PHP

Son bibliotecas/infraestructura para desarrollo web que proveen algunos/muchos módulos habituales y que, en muchos casos, están orientados a arquitecturas MVC para la implementación de la aplicación

Laravel
Symfony

CodeIgniter
Yii
CakePHP
Zend
PHPixie
PHP-MVC
FuelPHP
Slim
PHPixie
Phalcon
Drupal
...



● Laravel Aplicación ● Symfony Aplicación

25 nov 2018 9 ago 2020 24 abr 2022

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

46